

AgentProf: Semantic Profiling for AI Agents

Anonymous submission

Abstract

AI agents increasingly orchestrate long-running, multi-step activities involving thousands to millions of interactions with users, tools, and system resources. To improve quality, safety, and cost efficiency of AI agent, developers need to determine where failures concentrate, which workflows trigger unsafe effects, and which task categories consume the most budget. To answer similar questions in traditional systems software, profiling attributes resource consumption to responsible entities, such as code paths, by aggregation, rather than single-execution debugging or tracing. Yet existing agent observability tools are mainly for debugging and tracing instead of profiling. Agent behavior differs from code execution, creating two challenges for profiling: the responsible entities are semantic categories such as task intent and workflow phase, not code paths, and agent trajectories lack the stable identifiers and structural hierarchies that make attribution possible. We propose a *semantic operation stack model* that adapts profiling to agent trajectories. Uniform *operations* represent all agent activities, and *operation stacks* replace the runtime call stack, enabling hierarchical attribution at different levels of granularity from the same data. AGENTPROF implements this model as a profiler with pluggable algorithms for *intent attribution* and *stack construction*, compiling local agent trajectories into pprof-compatible profiles. On real trajectories and public datasets, our evaluation shows that semantic profiling effectively attributes cost and locates problems across agent trajectories.

Introduction

AI agents (Yang et al. 2024; Anthropic 2025; Xie et al. 2024) orchestrate multi-step activities that span two layers, high-level intent (prompts, LLM calls, tool invocations) and low-level system effects (process spawns, file and network I/O). As agents evolve from short, scripted workflows into complex systems that run for hours and days, with a large number of interactions each, production teams accumulate many such trajectories over weeks or months.

To improve agent quality, safety, and cost efficiency, developers need to analyze operational behavior across trajectories and interactions. Which categories of tasks consume

the most token budget? Where do failures concentrate across workflows? Which behavioral patterns trigger unsafe system effects? Answering these questions requires analysis and evaluation across many trajectories (Mazaheri and Mazaheri 2026; Lù et al. 2025), a task becoming costly for manual inspection or per-trajectory LLM evaluation (Zheng et al. 2023) at scale. In traditional systems software, profiling answers these questions by attributing resource consumption to responsible entities by aggregation, as distinct from single-execution debugging and tracing.

Existing agent tools provide rich tracing, tag/metadata aggregation, and population dashboards (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024; Datadog 2025; Laminar 2025). They rely primarily on run-local structure and application-supplied fields; they do not derive recurring semantic responsibility from heterogeneous agent histories and propagate it to downstream process, file, and network effects as alternative profiler hierarchies.

Adapting profiling to agent trajectories is challenging (§). In traditional software, the responsible entities are code paths. Profiling is straightforward because function names are stable and runtime call nesting provides the attribution hierarchy (§). Agent behavior differs fundamentally. To answer the questions above, the responsible entities need to be semantic categories such as task intent and tool operation, not code paths. Agent trajectories lack the two properties that make traditional profiling possible. Prompts are natural language, so two prompts expressing the same intent share no common string. Agent events have no runtime call-stack hierarchy because prompts, tool calls, and system effects are not nested by execution the way function calls produce stack frames: a sequence of prompts can be a task or multiple tasks, a task can be part of a bigger task.

Despite these differences, the fundamental profiling methodology transfers to agent trajectories: project resources onto responsible entities, assign stable tags, and attribute hierarchically. We propose a *semantic operation stack model* with two components. *Operations* are uniform records with string fields and additive measures that represent all agent activities (prompts, LLM calls, tool invocations, and system effects) without type-specific objects. *Operation stacks* provide hierarchical attribution, replacing the runtime call stack. Choosing different fields changes the attribution granularity: the same operations can be attributed by task category, by

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

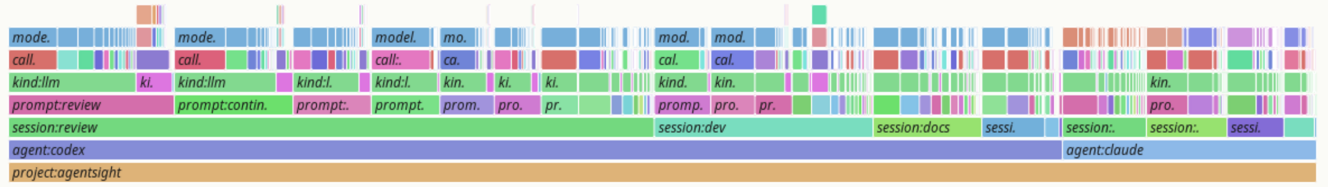
agentpprof tokens profile

prefix-merged flamegraph; width = count; total = 21899030768; drawn nodes = 940; hidden tiny nodes = 7331; depth = 7



agentpprof time profile

prefix-merged flamegraph; width = seconds; total = 2263796; drawn nodes = 865; hidden tiny nodes = 7474; depth = 10



agentpprof files profile

prefix-merged flamegraph; width = count; total = 46009; drawn nodes = 2051; hidden tiny nodes = 19126; depth = 7

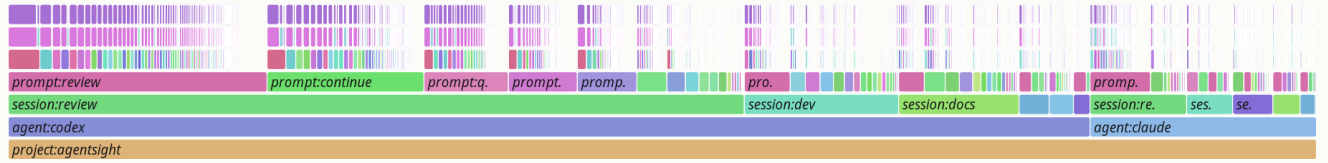


Figure 1: Semantic flame graphs of 325 real agent trajectories under three views. *Top (tokens)*: width represents token count. *Middle (time)*: width represents wall-clock duration. *Bottom (files)*: width represents file-operation count. All three are produced from the same operations by changing only the stack fields and weight function.

execution phase, by session, or by action type (Figure 1).

We implement this model in **AGENTPROF**, an offline profiler that compiles local agent trajectories into pprof-compatible profiles (Figure 2). Two pluggable algorithm frameworks enrich the model. *Intent attribution* derives stable, low-cardinality tags from natural-language fields via regex rules, a local LLM tagger, or unsupervised clustering. This gives agents the stable tags that traditional profiling gets for free from function names. *Stack construction* builds the attribution hierarchy, either from a user-specified field list or by automatically detecting boundaries between distinct work phases. On 325 real Codex and Claude trajectories (183,714 system observations), semantic profiling separates over 90% of system-effect cost that per-session views cannot attribute to task categories. Across three complete public benchmarks totaling 27,346 labeled steps, target-blind semantic profiles improve problem concentration over raw action: AP rises from 0.556 to 0.588 on AgentProcessBench, while inspection work falls from 46.29% to 41.57% at 80% recall on HINTBench and from 46.64% to 19.55% at 50% recall on TraceElephant. On the exact union of four public operation workloads, **AGENTPROF** builds a 27,765-operation seman-

tic profile in 1.17 s with 464.5 MiB maximum RSS, adding 18.2% time and 1.3% memory over a raw-action cost control.

This paper makes three contributions:

1. **Semantic operation stack model.** We identify the missing profiling layer in agent observability and propose a model of uniform operations and query-time operation stacks (§–§).
2. **System: AGENTPROF.** A profiler that implements this model with pluggable intent attribution and stack construction, producing pprof-compatible output (§).
3. **Evaluation.** We evaluate profiler output on 325 real trajectories, 15 mapped public trajectory families, and three complete independently annotated public benchmarks (§).

Background and Motivation

LLMs and AI Agents

A typical agent trajectory interleaves two layers of activity. At the intent layer, the agent receives a user prompt, issues LLM calls, and invokes tools (code execution, web search, file editing). Each tool invocation triggers system effects at

the lower layer: process spawns, file reads and writes, and network requests. A single trajectory may contain hundreds of such intent-effect cycles, and a production team accumulates many trajectories over time.

System Profiling

Profiling attributes resource consumption to responsible entities by aggregation, as distinct from single-execution debugging and tracing. In traditional software, the profiling pipeline works as follows: a profiler periodically samples execution state, attaches each sample to the current call stack, and merges samples with identical stacks. Flame graphs (Gregg 2011) and pprof (Google 2024b) call graphs visualize the result, where width or node size represents aggregate cost. Perfetto (Google 2024a) generalizes this with SQL-based analysis and derived events. These tools commonly begin with stable code identifiers and runtime stacks; pprof can also aggregate labels and promote `tagroot/tagleaf` values to pseudo-frames. Agent trajectories instead must first obtain stable semantic responsibility fields and connect them to downstream effects before such profile views exist.

Challenges for Agent Profiling

Existing tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) record agent events as per-execution span trees, effective for debugging a single run. AgentSight (Zheng et al. 2025) bridges intent and system-effect layers by correlating eBPF-captured system events with intercepted LLM traffic, but the resulting recordings still lack aggregate profiling.

Adapting profiling to agent trajectories faces two core challenges. The first is that the responsible entities differ. In traditional software, profiling attributes cost to code paths, but in agent trajectories the relevant entities are semantic categories such as task intent and workflow phase. Existing tools (OpenTelemetry 2024; Arize AI 2024b) can aggregate supplied attributes, but natural-language histories do not automatically provide stable responsibility categories: two prompts and tool invocations expressing the same intent may use different strings. The second challenge is that agent trajectories have no runtime hierarchy for attribution. In CPU profiling, the call stack determines attribution at every level: `malloc`'s cost belongs to `parse`, to `process`, and to `main` simultaneously. A sequence of prompts may constitute one task or several, and a task may be part of a larger workflow, but no runtime mechanism marks these boundaries or determines at which granularity to attribute cost. We quantify this in RQ1 (§).

Design Requirements for Agent Profiling

Traditional profiling relies on three mechanisms that the call stack provides automatically. Agent profiling needs all three but must achieve them without a call stack:

R1: Cross-layer resource projection. In traditional profiling, each sample is taken inside a call context, so resource consumption is automatically attributed to the code path that caused it. Agent activities span two layers: intent (prompts, LLM calls) and system effects (file I/O, process

spawns). The profiler must connect system-layer resource consumption to intent-layer semantic categories.

R2: Stable tags for folding. In traditional profiling, function names are stable identifiers that enable folding identical stacks. Agent prompts are natural language, so the profiler must derive short, stable, repeatable tags before folding.

R3: Hierarchical attribution. In traditional profiling, the runtime call stack provides the attribution hierarchy: function calls nest, and folding identical stacks produces the tree that profiles visualize. Agent events are not nested by execution, so the profiler must construct attribution hierarchies from the data.

Design

The three requirements (cross-layer resource projection, stable tags, and hierarchical attribution) drive the design. Operations (§) satisfy R1 by representing intent and system-effect activities in a single record with tag inheritance, so intent-layer tags propagate to the system effects they trigger. *Intent attribution* (§) satisfies R2 by deriving stable tags from natural-language fields. *Operation stacks* (§) satisfy R3 by replacing the runtime call stack with query-time field projections that attribute resources at multiple levels of granularity, so the same data supports multiple views without rebuilding the input. The profiling pipeline has four stages: (1) parse raw trajectories into operations, (2) enrich operations via intent attribution or mapping rules, (3) project each operation onto a stack frame sequence chosen at query time, and (4) fold operations with identical stacks, summing their weights.

Semantic Operation Stack Model

Because agent activities span both the intent and system-effect layers described in §, a profiler should not distinguish between them in the data representation. AGENT-PROF therefore represents every activity as a single abstraction, the *operation*, a weighted record with string fields and additive measures. Each operation carries string fields (`project`, `agent`, `session`, `prompt_tag`, `kind`, `model`, `path`, `domain`, `status`, ...) and additive measures (token count, duration, event count). A prompt, an LLM call, a tool invocation, a file read, a GUI click, and a process event are all operations with the same schema, distinguished only by which fields carry values.

An *operation stack* provides hierarchical attribution for agent trajectories, replacing the runtime call stack. In CPU profiling, the call stack determines which code paths share responsibility for a resource cost. An operation stack serves the same role: an ordered list of fields $[f_1, f_2, \dots, f_k]$ projects each operation o to a frame sequence $\langle o.f_1, o.f_2, \dots, o.f_k \rangle$, and operations with identical sequences are merged, summing their weights. Unlike a call stack, the fields are chosen at query time, not determined by execution. The same operations can therefore be attributed as a flat task summary, a per-session tree, a semantic phase/action hierarchy, or a trajectory using the dataset's own annotations, and changing the fields changes the attribution without changing the data. Sessions and spans are op-

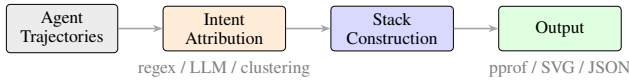


Figure 2: Data-flow and algorithm pipeline. Local agent histories enter through `agent-session` parsing; public datasets enter as operation JSONL. Both paths produce uniform operations. Intent attribution, stack construction, and folding are query-time algorithms applied identically to both sources.

tional operation fields, not fixed hierarchy levels, so the same data supports both debugging views (include `session`) and aggregate profiling views (exclude it). A view is a triple (φ, σ, w) : a predicate φ selects which operations participate, a stack function $\sigma = [f_1, \dots, f_k]$ maps each operation to its frame sequence, and a weight function w assigns a nonnegative measure. Four built-in views cover common questions: `tokens` (LLM calls weighted by token count), `time` (all timed operations weighted by duration), `files` (file operations weighted by event count), and `network` (network operations weighted by event count).

Algorithms

Intent attribution. Intent attribution maps natural-language prompts to short, stable, repeatable tags, giving agent trajectories the stable identifiers that traditional profiling gets for free from function names (§). The framework supports three pluggable backends. Regex rules are the production default: rules have the form `KIND:TAG=REGEX`, are evaluated in order (first match wins), and an AI Agent can iterate the regex with AGENTPROF until the unmatched rate drops below 5%, typically in 5–10 rounds. A local LLM tagger (e.g., a quantized 3B-parameter model via `llama.cpp`) generates a single-word tag per prompt with grammar-constrained decoding (forcing output to match a predefined token grammar) and caching, so AI Agent can help iterate the prompt as part of the configuration. Unsupervised clustering via TF-IDF and K-Means discovers natural prompt groups without predefined rules, feeding rule authoring. For structured trajectories where action types and quality annotations are explicit fields, mapping rules derive new fields from existing ones (e.g., `phase:navigate=type:(click|scroll|key)`).

Stack construction. Stack construction builds attribution hierarchies from available fields. When users supply an ordered list of fields, the projection is direct. When no field list is supplied, AGENTPROF induces operation stacks automatically. It scores each pair of adjacent operations for shifts in prompt content (via TF-IDF cosine similarity), changes in structured field values, and consistency of the resulting groups, then places boundaries where the combined score is highest. The induced stacks have variable depth, and a configurable depth cap controls the granularity. This mode produces coarser groups than hand-specified stacks but requires no domain knowledge, making it useful for initial exploration (§).

Implementation

AGENTPROF is an offline, post-hoc profiler implemented as a Rust CLI (~9.8K LOC, Figure 2). It reads Codex and Claude Code local JSONL history files and AgentSight (Zheng et al. 2025) recordings for system effects, and reconstructs prompts, LLM calls, tool invocations, and their triggered file and network effects. Output formats include pprof protobuf (compatible with `go tool pprof -http`), folded-stack text (compatible with `flamegraph.pl`), standalone SVG flame graphs, and JSON summaries.

Evaluation

We use three classes of data. *Real agent trajectories*: 325 readable local trajectories (198 Codex, 50 Claude, 77 Claude subagent) from a multi-month development project, containing 183,714 system observations. *Public annotated trajectories*: 15 families of real agent or human executions, totaling 47,590 operations through mapping rules. These cover web agents (WebLINX (Lù, Kasner, and Reddy 2024), Mind2Web (Deng et al. 2023), AgentTrek (Xu et al. 2024), WebShop (Yao et al. 2022)), API agents (API-Bank (Li et al. 2023), ToolBench (Qin et al. 2024), tau-bench (Yao et al. 2024)), coding agents (SWE-agent (Yang et al. 2024)), and mobile/GUI agents (AndroidCtrl (Li et al. 2024), GUI-Odyssey (Lu et al. 2025), ScaleCUA (Liu et al. 2025)). Four of these mapped families provide structured quality or boundary annotations: AgentRewardBench (Lù et al. 2025), SATraj-OS (Chen et al. 2026b), AgentNet (Wang et al. 2025), and OSWorld-Human (WukLab 2025). *Public problem-localization benchmarks*: AgentProcessBench, HINTBench, and TraceElephant provide the independent targets for RQ2. HINTBench reports 629 benchmark trajectories; the current official test snapshot enumerates 536 trajectories, and we use every one of them after selecting the field order on the separate 80-trajectory validation snapshot.

RQ1: Does Semantic Profiling Improve Resource Attribution?

We test whether semantic tags improve resource attribution beyond what traditional tag-free tools provide. We run a semantic-axis ablation on 325 real Codex/Claude trajectories (183,714 system observations). We progressively add tag axes (no tags, session only, prompt only, both) and measure the mixed-weight percentage, the fraction of system-effect cost in groups that mix observations from multiple task categories. A lower mixed-weight percentage means fewer effects from different prompt-tag categories remain in the same responsibility group. The residual percentage measures weight in groups where the majority category accounts for less than 90% of the group, capturing partial mixing.

Figure 3 shows the result. Without semantic tags, nearly all system-effect cost falls into mixed groups: a flat cargo test counter shows 2,903 executions but cannot distinguish whether they came from `review`, `debug`, `refactor`, or `test` prompts. The prompt tag is the decisive axis: it drops both mixed weight and residual by more than half and nearly doubles the number of unique stacks,

because it separates observations that share the same system call but originate from different task intents. The session tag alone barely helps because trajectories typically contain multiple intent phases.

A tag-permutation test (1,000 shuffles, $p = 0.001$) confirms the separation is not random. RQ3 separately tests semantic-tag accuracy against independent, matched annotation constructs. These results show that prompt-level semantic fields materially refine responsibility views beyond tag-free aggregation.

Beyond tag separation, we test whether choosing different field selections for the operation stack produces views that flat grouping cannot. We take the same 13,265 operations from the four families with ground-truth annotations and fold them with five different field selections, counting how many distinct groups each produces. The same operations fold into 9 dataset groups, 57 phase groups, 226 semantic groups, 455 action groups, or 3,757 per-session groups by changing only the stack fields. A flat GROUP BY on a single tag cannot produce these views because different stack depths answer different questions (budget by task vs. duplication by action vs. per-session detail). The same model also covers all 15 public trajectory families without type-specific objects. This shows that operation stacks provide a multi-resolution view that flat grouping cannot: one model answers questions at dataset, phase, action, and session granularity by changing only the field selection.

A profiler that supports only one weight function (e.g., token count) might miss bottlenecks visible under a different metric. We test whether different weight functions produce redundant or distinct rankings on the same trajectories. The `time` and `tokens` views of the same 325 trajectories share only 7/10 top prompt categories (Spearman $\rho = 0.623$). `network`-tagged prompts rank 8th by time (I/O wait) but 93rd by tokens. The rankings are not redundant: a single weight function would miss bottlenecks that are prominent under a different metric, confirming that multi-view profiling is necessary.

Finally, we test whether the profiler can produce useful groups without any user-specified fields. We run the automatic stack-induction mode (§) on six annotation tasks from the mapped-family corpus and compare the resulting group count and localization quality against flat summaries and per-session grouping. Induced stacks produce a median of 12 groups (versus 285 for per-session and 1 for flat), with 4/6 tasks generating variable-depth stacks. The induced stacks achieve lower average precision (AP, the area under the precision-recall curve when groups are ranked by problem density) than hand-specified stacks (median 0.276 vs 0.312) because they use only visible fields, but they reduce inspection work to 65.3% versus flat summaries without requiring any user configuration. Automatic induction therefore provides a useful zero-configuration baseline: it loses some precision versus hand-specified stacks but still substantially outperforms flat summaries.

Figure 1 shows the resulting flame graphs for the `tokens` and `files` views, illustrating that changing only the stack fields and weight function produces different aggregates from the same operations.

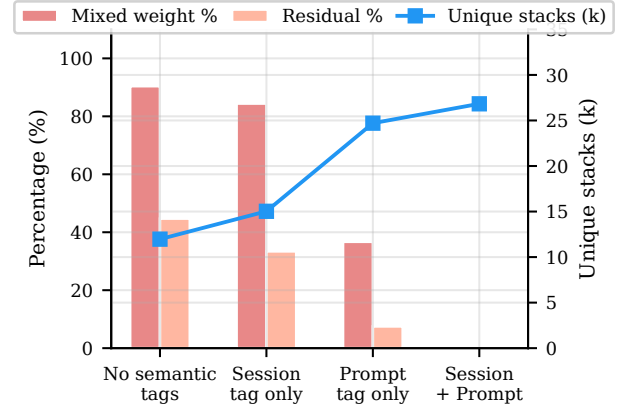


Figure 3: RQ1: Semantic-axis ablation on 183,714 system observations from 325 real trajectories. Bars show mixed weight and residual percentages (left axis); the line shows unique stacks (right axis). Adding prompt tags reduces mixed weight from 90.4% to 36.7% while increasing unique stacks from 12k to 25k. Session tags alone barely help (84.4%).

Table 1: Target-blind RQ2 localization across public workloads. Work is the fraction of atomic steps inspected at the stated macro recall.

Workload	Steps	AGENTPROF	Raw
AgentProcess	8,509	AP 0.588	AP 0.556
HINT	12,877	Work@80 41.57%	46.29%
TraceElephant	5,960	Work@50 19.55%	46.64%

RQ2: Does Profiler Output Correspond to Real Problems?

RQ2 tests whether a target-blind profile concentrates independently annotated failures or risky steps into groups that a developer can inspect early. We evaluate complete public workloads from AgentProcessBench (Fan et al. 2026), HINTBench (Wang et al. 2026b), and TraceElephant (Chen et al. 2026a). HINTBench development labels select among 24 fixed field orders; its reported test labels and all AgentProcessBench and TraceElephant targets remain held out from tag, stack, and ranking construction and are read only by the final scorer.

AgentProcessBench contains 1,000 trajectories from BFCL, GAIA, HotpotQA, and τ -bench with consensus human labels for all 8,509 assistant steps. Using the same released target-blind risk signal, the semantic operation stack raises equal-family macro AP from 0.556 to 0.588; the paired gain is 0.0315 with a 95% interval of [0.0151, 0.0535]. A matched refinement test that preserves every raw-action leaf and semantic subgroup size gives $p = 0.00995$, showing that the gain comes from the semantic assignment rather than merely creating more groups. At a 30% inspection budget, recall rises from 35.85% to 43.52%.

The independent workloads show the same practical effect in different inspection regimes. On all 536 released HINT-

Table 2: RQ4 profile-construction cost. Times are three-run medians; RSS is the largest observed semantic-profile peak.

Workload	Ops	Sem. (s)	Raw (s)	RSS (MiB)
AgentReward	729	0.04	0.03	19.0
Satraj	4,285	0.17	0.14	73.5
OSWorld	6,010	0.25	0.21	108.4
AgentNet	16,741	0.71	0.58	278.4
Union	27,765	1.17	0.99	464.5

Bench test trajectories, AGENTPROF reaches 80% macro recall after 41.57% of the steps, compared with 46.29% for raw action and 57.93% for the source-native order. On all 220 TraceElephant real failed executions, it reaches 50% macro decisive-step recall after 19.55% of the steps, versus 46.64% for raw action; at a 20% budget it recovers 52.57% versus 23.79%.

These results answer RQ2 positively: operation stacks expose recurring groups that are enriched for independently labeled problems and surface them earlier than raw-action inspection across three public workloads.

RQ3: How Accurate Are the Tags?

RQ1 and RQ2 assume that semantic tags (the tags derived by intent attribution or mapping rules) are useful. RQ3 tests whether these semantic tags are accurate by comparing them against ground-truth annotations from the benchmark datasets.

We test the positive hypothesis that a fixed, target-blind tagger or mapping assigns accurate and stable task, phase, action, and boundary identities on unseen agent and task families. Each identity is scored against an independent annotation of the same construct: task tags against task identity, action tags against action identity, phase tags against semantic phase, and phase transitions against human sequence boundaries. The tagger and mapping remain fixed before evaluation, and downstream profile aggregates are reconstructed from the held-out identities while preserving the original additive weights.

RQ4: What Is the Profiling Cost?

AGENTPROF is an offline, post-hoc profiler, so it adds zero overhead to the agent’s runtime. The profiling cost is the time and resources spent after the session ends. We measure release `agentpprof 0.2.37` on four complete existing public operation workloads and their exact union. For each input, we run a fixed semantic hierarchy and an identical-input raw-action cost control three times; both execute the full parse, stack-construction, folding, and serialization path.

Both time curves are monotonic across the five natural input sizes. The semantic curve has a descriptive slope of 0.0422 ms per operation ($R^2 = 0.9997$), reaching 23,731 operations/s on the union. At that largest input, semantic construction adds 180 ms (18.2%) and 6.0 MiB maximum RSS (1.3%) over raw action.

Field derivation is separately cacheable. In one identical-input predecessor AgentFlame pair over eight real Codex

sessions, the clean construction made 60 local-model calls and took 1.64 s; the cached reconstruction obtained 76/76 hits, made no model calls, and took 0.11 s. This pair supports the shared tag-cache mechanism’s elimination of repeated field-derivation calls; it is not a repeated timing estimate for the current `agentpprof` binary. Together, these results answer RQ4: profile construction is practical and predictable over the tested range, and cached derivation avoids repeated model work.

Related Work

Agent observability tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024; Arize AI 2024b; Dong, Lu, and Zhu 2024; Balusu 2026; Wang et al. 2026c; Datadog 2025; Laminar 2025) combine per-execution traces with tag, metadata, signal, and metric aggregation. AGENTPROF complements these views by deriving recurring semantic responsibility fields from heterogeneous histories and propagating them to additive cross-layer effects before constructing query-time profile hierarchies. AgentRx (Barke et al. 2026), TELBench (Wang et al. 2026a), AgentAtlas (Mazaheri and Mazaheri 2026), TrajAD (Liu et al. 2026), and AgentFixer (Mulian et al. 2026) perform per-trajectory fault localization. AGENTPROF complements all of these with category-level aggregate profiling across trajectories.

Conclusion

Agent observability needs profiling, not only debugging. AGENTPROF shows that the semantic operation stack model, with pluggable intent attribution and stack construction, brings hierarchical attribution to agent trajectories. Across 325 real trajectories, semantic fields separate system effects that tag-free and per-session views leave mixed and expose distinct token, time, and system-effect responsibility views. Across AgentProcessBench, HINTBench, and TraceElephant, target-blind operation stacks improve annotated-problem concentration over raw-action inspection at the workloads’ reported operating points. On 27,765 public operations, the release profiler builds the complete semantic profile in 1.17 s with 464.5 MiB maximum RSS, adding only 18.2% time and 1.3% memory over raw action. Together, these results establish cross-trajectory semantic profiling as a complement to run-local debugging and tracing.

References

- Anthropic. 2025. Claude Code: An Agentic Coding Tool. <https://code.claude.com/docs/en/overview>.
- Arize AI. 2024a. Arize Phoenix. <https://arize.com/docs/phoenix>.
- Arize AI. 2024b. OpenInference Specification. <https://arize-ai.github.io/openinference/spec/>.
- Balusu, K. C. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware ’26)*.
- Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent

- Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475.
- Chen, M.; Wang, J.; Mu, F.; Wang, Y.; Liu, Z.; Feng, H.; and Wang, Q. 2026a. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-Based Multi-Agent Systems. arXiv preprint arXiv:2604.22708.
- Chen, X.; Yin, Z.; He, S.; Huang, B.; Lei, S.; et al. 2026b. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230.
- Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.
- Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.
- Dong, L.; Lu, Q.; and Zhu, L. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv preprint arXiv:2411.05285.
- Fan, S.; Ye, X.; Huo, Y.; Chen, Z.-Y.; Guo, Y.; Yang, S.; Yang, W.; Ye, S.; Chen, J.; Chen, H.; Cong, X.; and Lin, Y. 2026. AgentProcessBench: Diagnosing Step-Level Process Quality in Tool-Using Agents. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.
- Google. 2024a. Perfetto Trace Processor. <https://perfetto.dev/docs/analysis/trace-processor>.
- Google. 2024b. pprof. <https://github.com/google/pprof>.
- Gregg, B. 2011. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- Laminar. 2025. Signals: Structured Event Extraction from Traces. <https://docs.lmn.ai/signals/introduction>.
- LangChain. 2024. LangSmith Observability. <https://docs.langchain.com/langsmith/observability>.
- Langfuse. 2024. Langfuse Observability. <https://langfuse.com/docs/observability/overview>.
- Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 3102–3116.
- Li, W.; Bishop, W. E.; Li, A.; Rawles, C.; Campbell-Ajala, F.; Tyamagundlu, D.; and Riva, O. 2024. On the Effects of Data Scale on UI Control Agents. arXiv preprint arXiv:2406.03679.
- Liu, Y.; Zhang, C.; Han, Z.; Liu, H.; Wang, Y.; Yu, Y.; Wang, X.; and Yin, Y. 2026. TrajAD: Trajectory Anomaly Detection for Trustworthy LLM Agents. arXiv preprint arXiv:2602.06443.
- Liu, Z.; Xie, J.; Ding, Z.; Li, Z.; Yang, B.; et al. 2025. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. arXiv preprint arXiv:2509.15221.
- Lu, Q.; Shao, W.; Liu, Z.; Meng, F.; Li, B.; Chen, B.; Huang, S.; Zhang, K.; Qiao, Y.; and Luo, P. 2025. GUIOdyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
- Lù, X. H.; Kasner, Z.; and Reddy, S. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. arXiv preprint arXiv:2402.05930.
- Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. arXiv preprint arXiv:2504.08942.
- Mazaheri, P.; and Mazaheri, K. 2026. AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. arXiv preprint arXiv:2605.20530.
- Mulian, H.; Zeltyn, S.; Levy, I.; Galanti, L.; Yaeli, A.; and Shlomov, S. 2026. AgentFixer: From Failure Detection to Fix Recommendations in LLM Agentic Systems. In *Proceedings of the 1st International Workshop on Agentic Engineering (AGENT '26, co-located with ICSE)*.
- OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. <https://github.com/open-telemetry/semantic-conventions-genai>.
- Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; Zhao, S.; Hong, L.; Tian, R.; Xie, R.; Zhou, J.; Gerber, M.; Li, D.; Liu, Z.; and Sun, M. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- Wang, J.; Feng, Z.; Wu, J.; Li, R.; Xie, Q.; Ren, Y.; Zhu, H.; Han, X.; Meng, F.; Feng, J.; and Liu, J. 2026a. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. arXiv preprint arXiv:2606.02060.
- Wang, J.; Hou, J.; Wang, F.; Jian, P.; Bao, C.; and Lv, Z. 2026b. HINTBench: Horizon-Agent Intrinsic Non-Attack Trajectory Benchmark. arXiv preprint arXiv:2604.13954.
- Wang, X.; Wang, B.; Lu, D.; Yang, J.; Xie, T.; et al. 2025. AgentNet. <https://huggingface.co/datasets/xlangai/AgentNet>.
- Wang, Y.; Zhang, J.; Cai, T.; Liu, Z.; Sun, Q.; Sun, Z.; Wu, Z.; Dong, M.; Zheng, M.; Yin, X.; and Zhu, Y. 2026c. From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents. arXiv preprint arXiv:2606.04990.
- WukLab. 2025. OSWorld-Human. <https://github.com/WukLab/osworld-human>.
- Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shi, D.; Tong, J.; Lu, K.-W.; Garg, V.; Wang, Y.; Dai, Z.; Li, F.; Bisk, Y.; and Yu, T. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.
- Xu, Y.; Lu, D.; Shen, Z.; Wang, J.; Wang, Z.; Mao, Y.; Xiong, C.; and Yu, T. 2024. AgentTrek: Agent Trajectory

Synthesis via Guiding Replay with Web Tutorials. arXiv preprint arXiv:2412.09605.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022. Web-Shop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv preprint arXiv:2406.12045.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Zheng, Y.; Hu, Y.; Yu, T.; and Quinn, A. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems (PACMI '25, co-located with SOSPP)*.